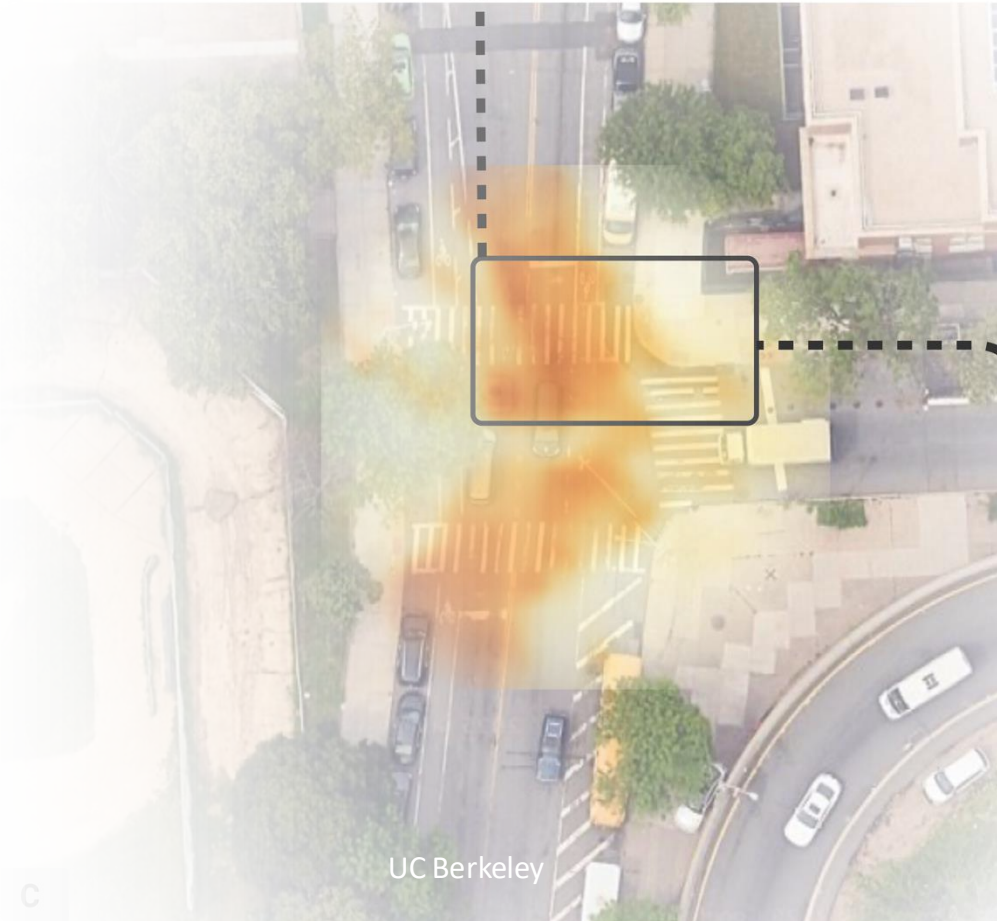
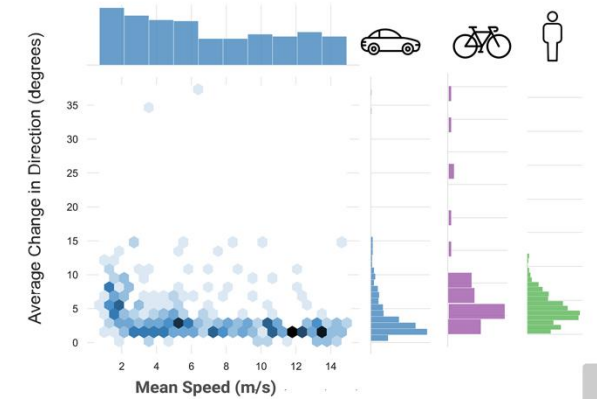
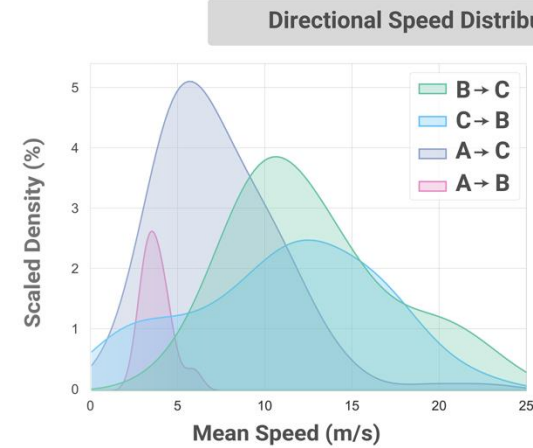
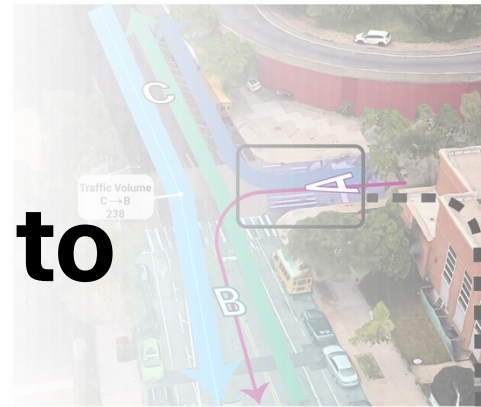


# Introduction to Urban Data Analytics

Fall 2026

Maryam Hosseini

City & Regional Planning



UC Berkeley

C

B

D

# Project Proposal

---

Think of the proposal as your opportunity to:

- Select a topic that interests your group
- Practice **problem definition** using the structured steps we have discussed
- Translate a real-world question into a **quantifiable, bounded research question**
- Break a complex problem into smaller, computationally tractable pieces
- Get early feedback from the teaching team to sharpen and de-risk your project

# Project Proposal

---

- Each group submits **one** proposal, either to MTC track or to the main track. Both will be graded by our team, **MTC does not grade you.**
- Proposal is a **checkpoint**, not a final commitment
- The plan you outline in your proposal is expected to evolve as you learn more

# Python Basics

---

# Types Recap

---

- Python has five basic types of values to represent data:
  - Integers are whole numbers. *int()*
  - Floats are decimal numbers. *float()*
  - Strings are textual data surrounded by quotes. *str()*
  - Booleans are `True` or `False` values that represent the answer to questions. *bool()*
  - **New:** `None` represents the specific absence of a value. *None*

# None Type (None)

---

- The None type is a special type that has **only one value**.
- None represents the absence of value.
- The None type has a capital N and no quotes

# Data Types in Python

---

|         | Data Type | Definition                                                                      | Example            |
|---------|-----------|---------------------------------------------------------------------------------|--------------------|
| Integer | int       | Integer numbers, whole numbers without a fractional part.                       | 42, -5, 0          |
| Float   | float     | Floating-point numbers, numbers with decimal points.                            | 3.14, -0.01, 0.0   |
| String  | str       | Strings, sequences of characters enclosed in single or double quotes.           | 'hello' , "Python" |
| Boolean | bool      | Boolean values, representing True or False.                                     | True, False        |
| None    | None Type | Represents the absence of a value, such as a rating that has not been assigned. | None               |

# Truthiness

---

- **Truthiness:** The ability to use any type of value as a conditional.
- Any expression can be evaluated as a conditional.
- *How* it's evaluated depends on its type.
- For `int` and `float`, any non-zero value is considered true.
- For `str` any non-empty string is considered true.
- The `None` type has no true values.

# Truthiness

**Truthiness:** The ability to use any type of value as a conditional.

| Type    | Falsy | Truthy           |
|---------|-------|------------------|
| Boolean | False | True             |
| Integer | 0     | Non-Zero Numbers |
| Float   | 0.0   | Non-Zero Numbers |
| String  | ""    | Non-empty string |
| None    | None  |                  |

# Control Flow

---

Conditionals & Loops



# Conditionals



# if

# Comparison Operators

---

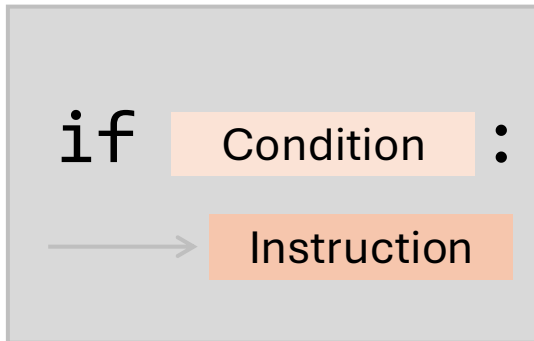
**Return Boolean:**

True

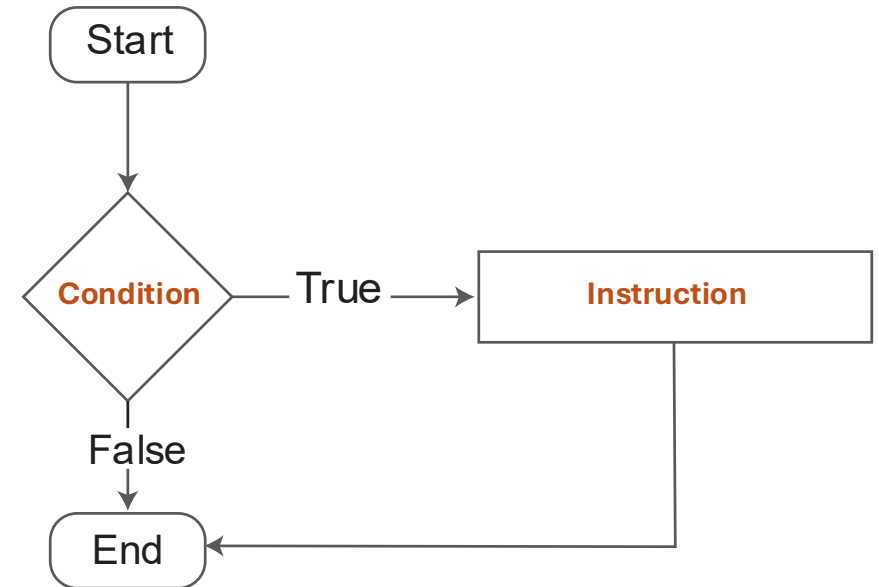
False

| Operation     | Meaning                 |
|---------------|-------------------------|
| <             | strictly less than      |
| <=            | less than or equal      |
| >             | strictly greater than   |
| >=            | greater than or equal   |
| ==            | <b>equal</b>            |
| !=            | not equal               |
| <b>is</b>     | <b>object identity</b>  |
| <b>is not</b> | negated object identity |

# Conditional



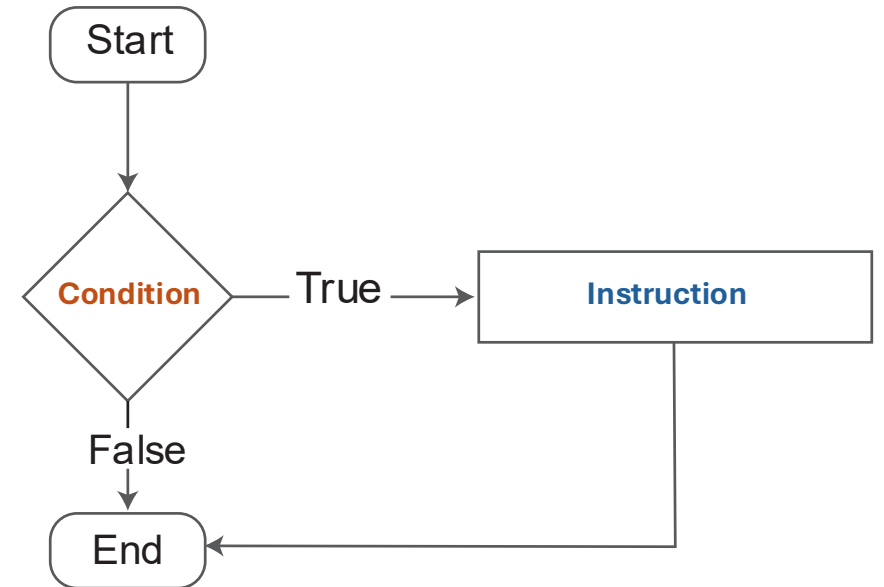
**if** followed by a **condition** and **:**  
Next line **one level indentation** (→)  
and our **instruction** of what to do  
if the condition evaluates to **True**



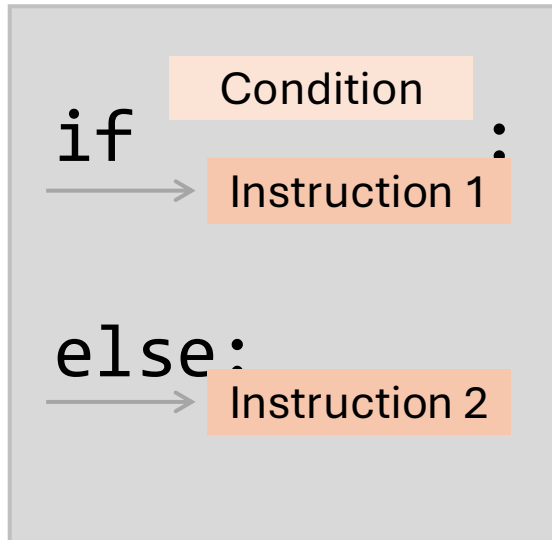
# Conditional

```
size = input("enter your file size")
```

```
if int(size) > 500:  
    print("Your file is too large")
```

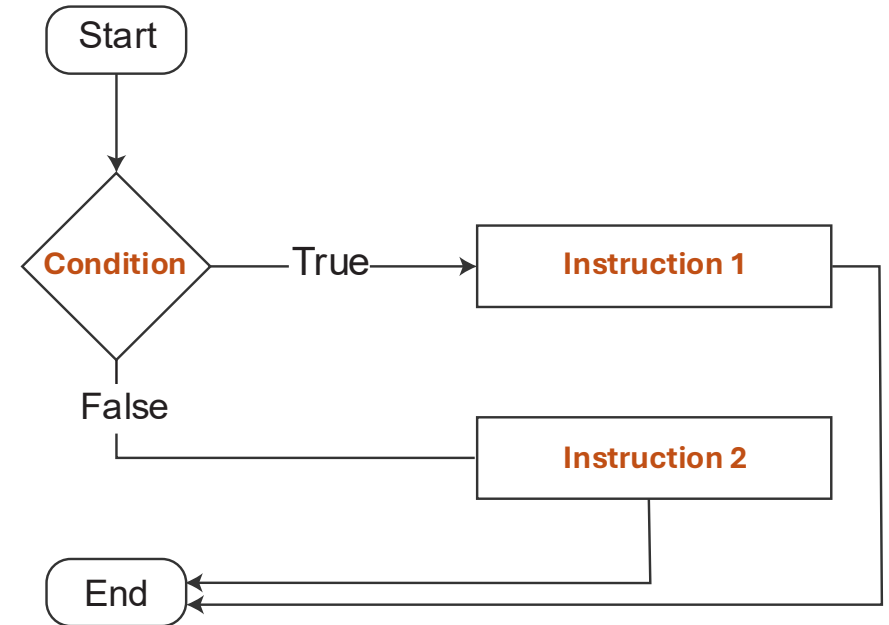


# Conditional



**if** followed by a **condition** and **:**  
Next line **one level indentation** (→)  
and our **instruction** of what to do  
if the condition evaluates to **True**

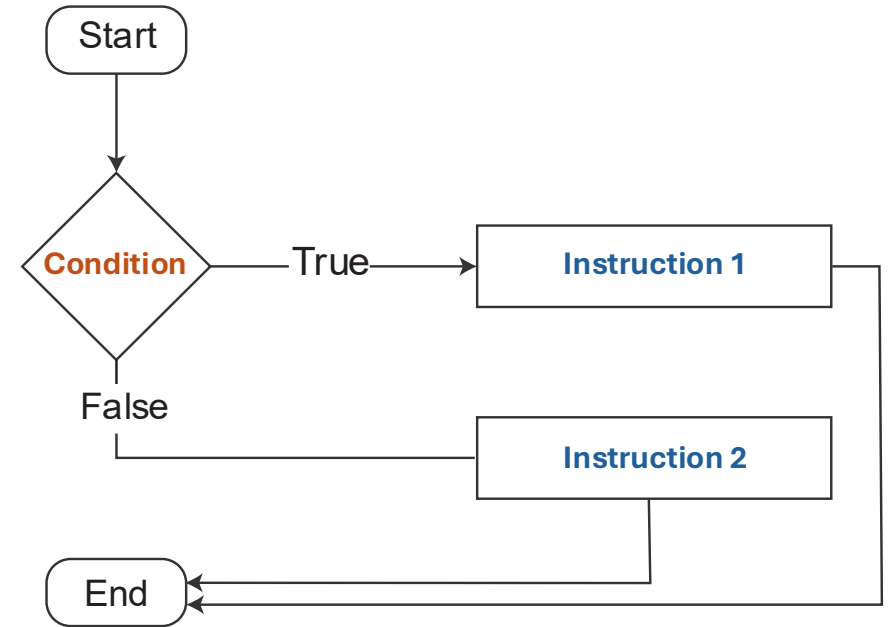
**else** tells Python what to do when  
other conditions do not evaluate to  
True



# Conditional

```
number = input("enter your number")
```

```
if int(number) % 2 == 0:  
    print("Even Number")  
else:  
    print("Odd Number")
```



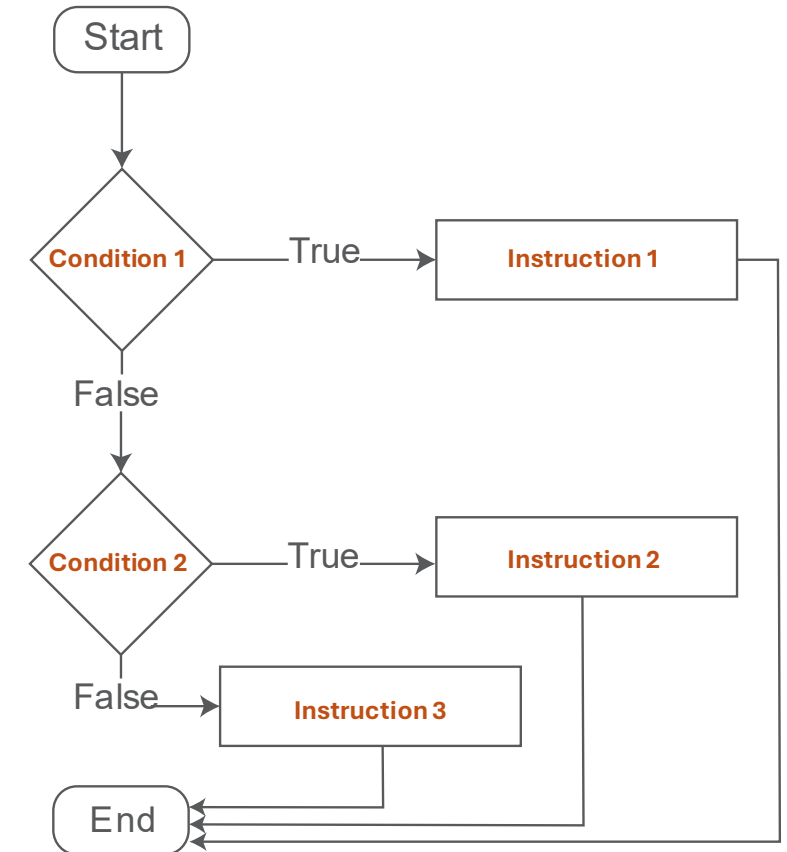
# Conditional

```
if Condition 1 :  
    → Instruction 1  
elif Condition 2 :  
    → Instruction 2  
else:  
    → Instruction 3
```

**if** followed by a **condition** and **:**  
Next line **one level indentation** (→)  
and our **instruction** of what to do  
if the condition evaluates to **True**

**elif** has a similar construct to **if**, setting  
another condition in case previous  
conditions do not evaluate to **True**

**else** tells Python what to do when other  
conditions do not evaluate to **True**



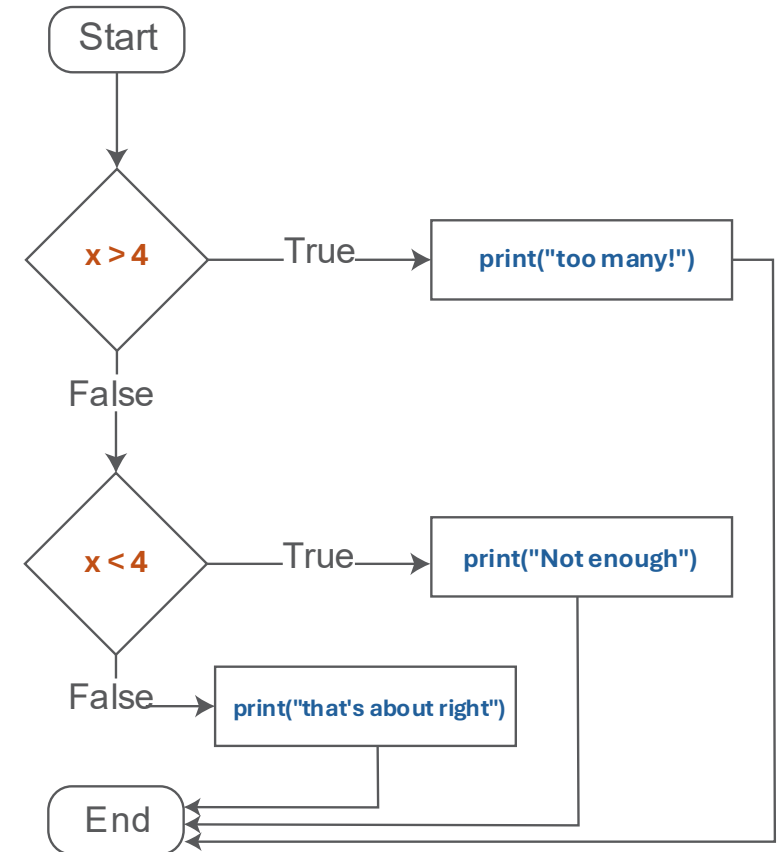
# Conditional

```
if Condition 1 :  
    → Instruction 1  
elif Condition 2 :  
    → Instruction 2  
else:  
    → Instruction 3
```

**if** followed by a **condition** and **:**  
Next line **one level indentation** (→)  
and our **instruction** of what to do  
if the condition evaluates to **True**

**elif** has a similar construct to **if**, setting  
another condition in case previous  
conditions do not evaluate to **True**

**else** tells Python what to do when other  
conditions do not evaluate to **True**



# Take Five





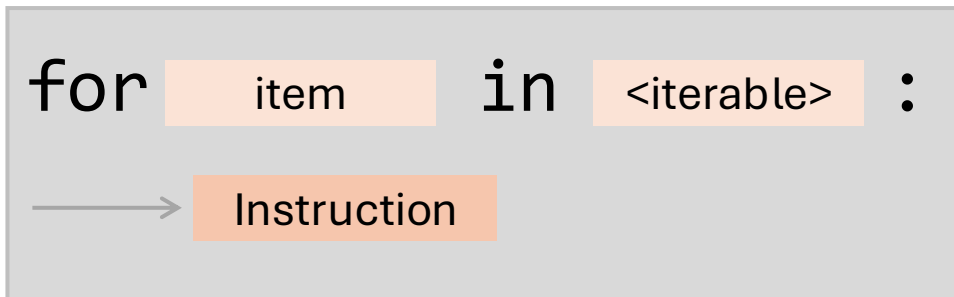
# Loops



for

# for Loop

for loop allows you to iterate over iterables (like a list) and access each item in it.



**Item** is the **variable** that will be automatically set to each of the objects in our iterable.

1. Type the word **for**
2. Make a new variable, named the *iteration variable*
3. Type the word **in**
4. Type the name of the iterable (e.g., list) you want to iterate over.
5. Put a colon ( **:** )
6. Put statements inside the body of the loop, indented **with 4 spaces**

# for Loop

1. Type the word **for**
2. Make a new variable, named the *iteration variable*
3. Type the word **in**
4. Type the name of the iterable (e.g., list) you want to iterate over.
5. Put a colon ( **:** )
6. Put statements inside the body of the loop, indented **with 4 spaces**

```
2 items = [  
3     0.23,  
4     5,  
5     "Berkeley",  
6     True,  
7     [1, 2, 3],  
8     (10, 20),  
9 ]  
10  
11 # What are the types of objects I have in my list?  
12 # Without for loop  
13  
14 print(items[0], type(items[0]))  
15 print(items[1], type(items[1]))  
16 print(items[2], type(items[2]))  
17 print(items[3], type(items[3]))  
18 print(items[4], type(items[4]))  
19 print(items[5], type(items[5]))  
20  
21  
22 print('*'*20)  
23  
24 #with for loop  
25 for item in items:  
26     print(item, type(item))
```

# Iteration Variable

- Sometimes referred to as the *iteration target*.
- Two critical facts to remember:
  - The `for` loop creates the iteration variable when it executes.
  - The type of the iteration variable is the same as the element type of the list.
- Notice the iteration variable names in the `for` loops code.

```
for a_temperature in temperatures:  
    pass  
  
for a_book in books:  
    pass  
  
for a_name in names:  
    pass  
  
for a_fruit in fruits:  
    pass
```

# Iteration Variable

---

What is the **type** of the *iteration variable* in this code?

```
digits = ["9", "3", "6"]  
for digit in digits:  
    print(digit)
```

- a) str
- b) int
- c) list[str]

# Iteration Variable

---

What is the **type** of the *iteration variable* in this code?

```
digits = ["9", "3", "6"]  
for digit in digits:  
    print(digit)
```

- a) **str**
- b) **int**
- c) **list[str]**

The element of the list are **str**,  
so the iteration variable is also a **str**.

# My Places!

---

| Resources                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------|
| <b>ACTIVITY</b> <ul style="list-style-type: none"><li>• Your Places, W04 S1: Filtering Your Places <a href="#">↗</a></li></ul> |



Follow the link to DataHub and let's do some coding!